Week 4: lists, matrices, sets, tuples and dictionaries

MIS41110 Programming for Analytics · 2026/27 · Week 4 lecture notes

These notes cover week 4's lecture, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

Where we started

Slides, pages 4, 6 and 7.

Last week's register worked, but it confessed two pains. The order was a single running total, so when a customer asked what they were paying for, we could not list the items: we had thrown them away and kept only the sum. And `price_of` was an if/elif chain that grew by two lines every time a product joined the menu, two lines to say one thing: this name has this price. Six products already take fifteen lines, a real menu would run to pages, and every lookup tests the products one by one, in order, until one matches. Both pains have one diagnosis: we were storing collections of things in tools built for single values. A running total is one number pretending to be a list of items; a chain of ifs is code pretending to be a lookup table. This week Python gives us four collection types, and both debts die.

Poll. How many times does `for x in range(7, 10): print(x)` run: 3, 9, 24, or once? Three: the body runs once for each value the range hands it, 7, 8 and 9, and the range stops before 10, because the stop number is never included. 24 is 7 + 8 + 9, the sum of the values, not the number of passes. Keep the stop rule close: slices follow it too, a few pages on.

Recap. Loops, from week 3: tracing one line at a time, and how a `while` loop stops. A question that tests both: the loop must run until `count` reaches 5, so what fills the hole in `while count ___ 5:`? Two answers are right, `<` and `!=`. Tracing means you play the computer: take one line, do what it says, write down what changed, then the next line. Here is `<`, with `count` starting at 0 and a body that adds 1, traced the way the slide does it:

Line that runs	`count`	The check
`count = 0`	0	before the loop
`while count < 5:`	0	`0 < 5` is `True`: into the body
`count = count + 1`	1	the body
`while count < 5:`	1	`1 < 5` is `True`
`count = count + 1`	2	the body
three more passes	3, 4, 5	
`while count < 5:`	5	`5 < 5` is `False`: out of the loop
`print(count)`	5	prints 5

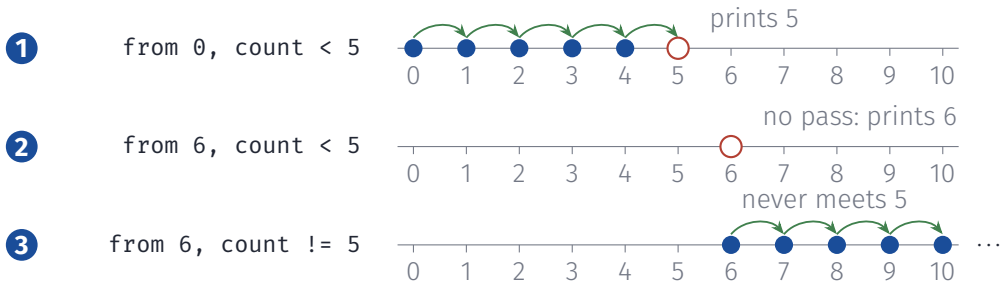
Back at the `while` line the check runs again, on every pass: that is the part people skip. When `5 < 5` comes out false, Python moves to the line after the body, which prints 5. With `<=` the check at 5 is still true, one more pass runs, and it prints 6: the off-by-one this question is built to catch. Doing this in your head is fine; writing it down helps a lot. For the exam, the slide's routine: run each line in order, write down what changed, stop at the `False`.

Look at where `count = 0` sits: before the loop, once. A `count = 0` inside the body would reset it on every pass, and the loop would never reach 5.

How do you read the `while` line itself? As "as long as": as long as the condition is true, run the body again. So when you write one, phrase the condition as what must stay true for the loop to go on. Sometimes that means turning your thinking around: "stop at 5" becomes "keep going while below 5".

`!=` stops at 5 too, which is why both are right. They are not the same loop, though. Start `count` at 6. With `<`, the check `6 < 5` is false at once, the body never runs, and it prints 6. With `!=`, the check `6 != 5` is true, `count` goes to 7, 8, 9, and never meets 5: the loop never stops. Which one is right depends on what the program needs, and when the start could be anything, `<` is the safer choice. And if a program of yours ever seems stuck, either you asked it to train the next ChatGPT or a loop never stops: press Ctrl-C in the terminal, then find the line that should change the condition.

Here are the three runs side by side. ❶ From 0, `<` runs the body at 0, 1, 2, 3 and 4, and the check fails when `count` reaches 5. ❷ From 6, `<` fails at the very first check: no pass at all, and it prints 6. ❸ From 6, `!=` keeps going, 7, 8, 9 and on, because `count` has already passed 5 and can never meet it. The check runs before every pass, and what it compares decides whether the loop can stop.



Week 3's follow-up is on the Q&A page, under Week 3.

1 · Lists

Slides, pages 8 to 16.

A list is a row of seats

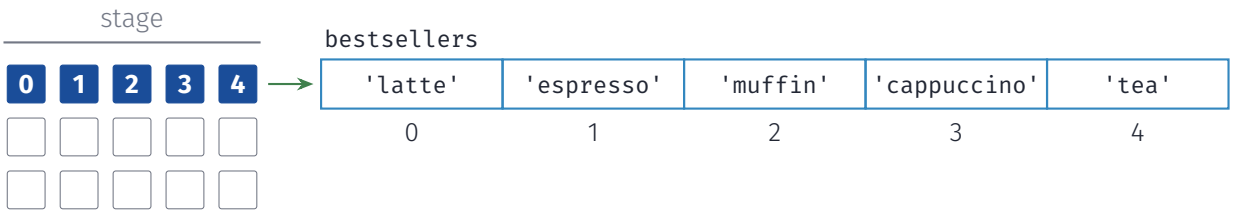
Until now I was hiding the name: every time I talked about the front row, I was talking about a list. Picture the front row of the lecture hall, lifted out and laid flat. Every seat holds one value and every seat has a number, starting at 0 at one end: that number is the **index**. Give the seat number and the seat tells you who sits there. A block of seats is a **slice**, and a string is a row too, one letter per seat. You have used a list since week 2 without its name: `sys.argv`, whose seat 0 holds the program's own name. Hold on to this picture: sets, tuples and dictionaries come back to the same room.

Making lists

You write values between square brackets, separated by commas, and a list holds anything, even values of different types, even other lists:

```
>>> bestsellers = ["latte", "espresso", "muffin", "cappuccino", "tea"]
>>> daily_revenue = [212.40, 305.90, 198.20, 240.00, 260.10]
>>> latte_card = ["latte", "Caffè Latte", 3.20, True]     # mixed types are fine
```

Here is the picture from above with `bestsellers` in its seats: the front row of the room, lifted out and laid flat. Every seat holds one value, and its number, counted from 0, is its index.



The tools you know still work: `type(bestsellers)` is `<class 'list'>`, `len(bestsellers)` is 5.

That freedom is also the danger. A list is permissive: it takes anything, and it never asks whether it makes sense. A `True` in a list of products? Python accepts it. Type the wrong variable name and add your revenue figures to the list of product names, and Python adds them without a word. Here is a list of ones, written four ways, changed seat by seat:

```
>>> ones = [1, "1", "I", True]       # an int, a str, a Roman numeral, a bool
>>> ones[1] = ones[1] * 2            # a string times 2 repeats it
>>> i = 0
>>> ones[i] = ones[i] * 2            # the index can be a variable
>>> print(ones)
[2, '11', 'I', True]
```

Python did not blink. A seat that held a number can hold text the next minute, so keep track of what your lists hold: nobody else will.

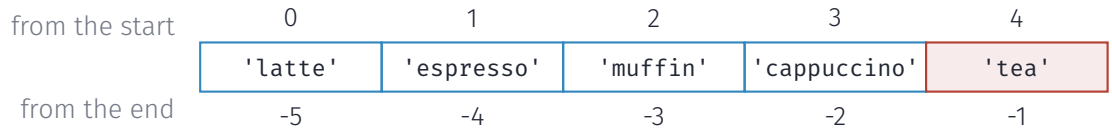
> **Good practice.** Keep each list to one kind of thing: all prices, or all names. A list that mixes a name, a price and a flag is a record in disguise, and records have better homes: a tuple when the fields are fixed (section 5), a dictionary when they have names (section 6).

Indexing and slicing

Python counts from zero, so the first element sits at index 0. An index is an offset from the start: zero steps from the start is the first seat, and seat 2 is the third. A negative index counts from the other end, so `[-1]` is always the last element, the same seat as `[4]` here, with no need for `[len(bestsellers) - 1]`.

```
>>> print(bestsellers)
['latte', 'espresso', 'muffin', 'cappuccino', 'tea']
>>> print(bestsellers[0])     # first element: Python counts from ZERO
latte
>>> print(bestsellers[-1])    # count from the END: the last element
tea
```

The same row, numbered from both ends: seat 4 and seat -1 are the same seat, the last one, and `[-1]` stays the last however long the row grows.

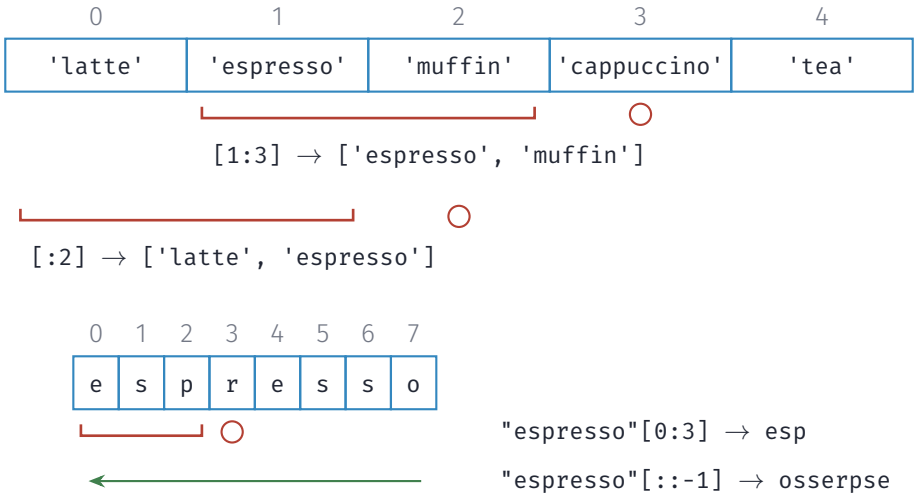


A slice runs from its first number up to, but not including, its second: the rule of `range` again, so `[1:3]` takes seats 1 and 2 and stops at 3. Leave the first number out and it starts at the beginning, so `[:2]` is `[0:2]`. A slice of a list is a list, a smaller row, and a slice of a string is a string. A slice has three slots, start, stop and step. Leave the step out and it is 1; leave start and stop out and the slice covers the whole row, walking in the step's direction. So `[::-1]` walks the whole row backwards, from the last seat to the first, with the same step that counted `range(5, 0, -1)` down in week 3.

```
>>> print(bestsellers[1:3])    # a SLICE: from 1 up to (not incl.) 3
['espresso', 'muffin']
>>> print(bestsellers[:2])     # from the start
['latte', 'espresso']
>>> print("espresso"[0:3])     # strings slice the same way
esp
>>> print("espresso"[::-1])    # the slice's THIRD slot: a step (-1 walks backwards)
osserpse
>>> daily_revenue[2] *= 1.1      # lists are mutable: change one element in place
>>> del bestsellers[1]          # delete by position (garbage collection collects)
```

Here are those slices on the row. A slice takes the seats from its start up to its stop and leaves the stop's seat out, the open circle, as `range` leaves out its stop. `[1:3]` takes seats 1 and 2. `[:2]` starts

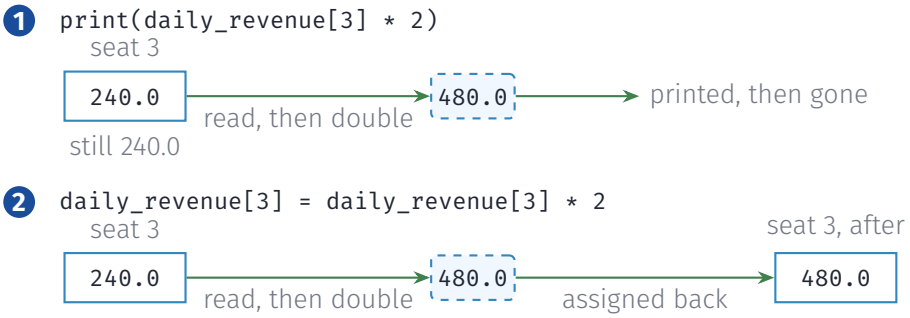
at the beginning and takes seats 0 and 1. A string's seats hold characters: `[0:3]` gives `esp`, and `[::-1]` walks the whole row from the last seat to the first.



Lists are **mutable**: you can change an element by assigning to its position. Reading a seat and changing it are two different things, though. `daily_revenue[3] * 2` reads the value, doubles it in a temporary spot and hands that to `print`; the seat keeps its 240.0. Only an assignment to the seat changes the list:

```
>>> print(daily_revenue[3] * 2)                # read, double, print: temporary
480.0
>>> print(daily_revenue[3])                    # the seat still holds 240.0
240.0
>>> daily_revenue[3] = daily_revenue[3] * 2    # assign back to the seat
>>> print(daily_revenue[3])
480.0
```

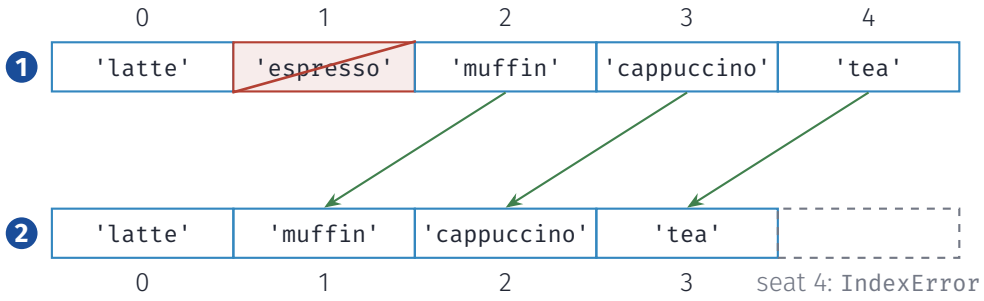
(1) Reading looks up the 240.0 in seat 3, doubles it in a temporary spot and hands the 480.0 to `print`; the seat never hears of it. (2) The assignment sends the 480.0 back into seat 3, and only then does the list change.



`del` takes a seat out of the row altogether: I don't like you, you disappear. Everything after it shifts one seat left, so the indexes change under your feet, and an index you wrote by hand can fall off the end. Before the `del` above, seat 4 held `"tea"`:

```
>>> print(len(bestsellers))     # one seat fewer since the del
4
>>> print(bestsellers[4])
IndexError: list index out of range
```

1 `del bestsellers[1]` takes seat 1 out of the row. 2 Every seat after it moves one place left, so `'muffin'` sits in seat 1 now and `'tea'` in seat 3. Seat 4 no longer exists, and asking for it is the `IndexError`.



Growing and shrinking

`append` adds one element at the end, a new seat at the end of the row. `pop` removes the last element and hands it back: think of a shelf of cookies, where every sale takes the last one off the shelf and into your hand. `[0] * 10` makes ten zeros, and `in` asks whether a whole element is there.

```
>>> order = []                      # an empty list...
>>> order.append("latte")           # ...grows at the end
>>> order.append("muffin")
>>> last = order.pop()              # pop(): remove the LAST element and hand it back
>>> zeros = [0] * 10                # a list of ten zeros, pre-sized
>>> print([0] * 3)
[0, 0, 0]
>>> print("latte" in order)         # membership: the same in as Week 3
True
```

Of everything on this page, learn `append`: every time the café takes another item, the order appends.

To add an element somewhere other than the end, say at position 2, there is a method, `insert`, on the toolbox page: give it the position, then the element. The hard way works too: cut the row in two with slices, and glue the pieces back together around the new element with `+`.

```
>>> shelf = ["latte", "espresso", "muffin"]
>>> shelf.insert(2, "cookie")                    # the method
>>> print(shelf)
['latte', 'espresso', 'cookie', 'muffin']
>>> shelf = ["latte", "espresso", "muffin"]
>>> shelf = shelf[:2] + ["cookie"] + shelf[2:]   # the hard way: cut, add, glue
>>> print(shelf)
['latte', 'espresso', 'cookie', 'muffin']
```

The new element goes in brackets because `+` glues a list to a list: without them, `shelf[:2] + "cookie"` raises `TypeError: can only concatenate list (not "str") to list`. Four verbs, then: `append` grows the row at the end, `pop` shrinks it from the end, `insert` adds at a position you name, and `del` removes at a position you name.

The order finally remembers

Pages 13 and 14. This is the first debt, paid. In `take_order` the order is now a list that starts empty, and every item the customer asks for is appended as one entry, so nothing is thrown away:

```
    order = []
    still_ordering = True
    while still_ordering:
        product = input("  Product (or 'done'): ").strip().lower()
        if product == "done":
            still_ordering = False
        elif product not in MENU:
            print("    Sorry, we don't sell that.")
        else:
            quantity = int(input("  Quantity: "))
            if quantity < 1 or stock[product] < quantity:
                print(f"    You can't buy {quantity} {product}(s): {stock[product]} left.")
            else:
                stock[product] = stock[product] - quantity
                order.append((product, quantity))
```

Two things in this code wait for later sections. `MENU` and `stock` are dictionaries, which section 6 explains. And the double parentheses in `append` bundle the product and its quantity into one order line, a tuple, which section 5 names. For now, read a tuple as a list that cannot change once it is written: the line says two lattes, and nothing later in the program can change the 2 inside it.

Run `cafe_w4.py`, type `Ada` as the cashier, choose `1`, give the customer name `ada`, order `latte` and `2`, then `muffin` and `1`, then `done`. The receipt lists every line:

```
======== The Daily Grind =========
latte        x  2     6.40
muffin       x  1     2.40
==================================
TOTAL                 8.80   (3 items)
```

Last week the bill was one number. Now every line is on the receipt, because every item was appended to a list.

The second page shows the same loop spelled another way, the one you will meet everywhere:

```
while True:              # loop forever...
    ...
    if product == "done":
        break            # ...and ESCAPE from inside
```

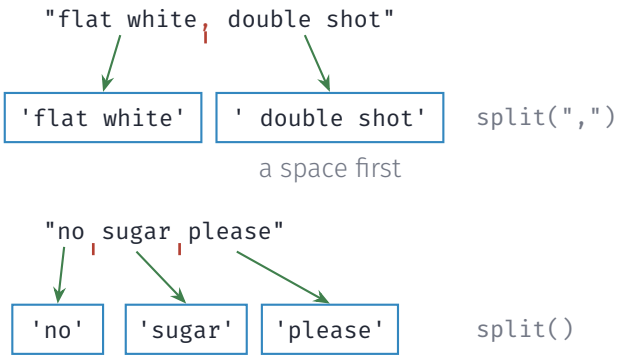
`while True:` loops forever, and `break` jumps out from inside the moment the customer says done. It does the same job as our `still_ordering` flag. The flag names the state, `break` jumps out, and both are correct. In nested loops, `break` leaves only the innermost one.

Lists and other iterables

`list` turns any sequence into a list: a string explodes into its characters, a range into its numbers. The star of the page is `split`, which cuts a string into a list of pieces, on a separator you choose or, with no argument, on whitespace. Every line you read from a file splits this way, and the case study in section 7 depends on it.

```
>>> list("latte")                    # a string, exploded into characters
['l', 'a', 't', 't', 'e']
>>> list(range(5))                   # a range, materialized
[0, 1, 2, 3, 4]
>>> "flat white, double shot".split(",")
['flat white', ' double shot']
>>> "no sugar please".split()        # default: split on whitespace
['no', 'sugar', 'please']
```

`split(",")` cuts at each comma and drops the comma, so the space after it stays at the front of the second piece: `' double shot'` starts with a space. With no argument, `split()` cuts at every run of spaces and keeps no space at all.



One promise from week 2 kept: `sys.argv` was a list all along, so `sys.argv[1:]` is every argument after the program's own name. The slide gives you the pair you will use for good, `pname, args = sys.argv[0], sys.argv[1:]`. Here it is on a pretend `sys.argv`, the list Python would hold for `python3 receipt.py latte 2`:

```
>>> argv = ["receipt.py", "latte", "2"]
>>> pname, args = argv[0], argv[1:]
>>> print(pname)
receipt.py
>>> print(args)
['latte', '2']
```

The first name gets seat 0, a string. The second gets the slice, a list, however many arguments there are. Section 5 explains how two names sit on the left of one `=`.

Split a review with a comma in it and the comma stays stuck to its word:

```
>>> review = "great espresso, but slow"
>>> words = review.split()
>>> print(words)
['great', 'espresso,', 'but', 'slow']
>>> words.split(",")
AttributeError: 'list' object has no attribute 'split'
```

`split` is a string method, and `words` is a list now, so there is no `split` to call. There are a few sensible roads. A clean one turns the order around and uses a loop: split on the comma first, then split each piece on the spaces, and glue the pieces together with `+`.

```
>>> words = []
>>> for part in review.split(","):
...     words = words + part.split()
...
>>> print(words)
['great', 'espresso', 'but', 'slow']
```

There is rarely one unique solution. Keep digging until something works, then keep the version you can read.

Predict, then run

Page 16's TRY IT asks for four predictions. Here they are, run:

```
>>> print(["first", "is", "best"][0])
first
>>> positions = ["first", "middle", "last"]
>>> print("midpoint:", positions[len(positions) // 2])
midpoint: middle
>>> print("-".join(["tic", "tac", "toe"]))
tic-tac-toe
>>> seasons = ["Spring", "Summer", "Autumn", "Winter"]
>>> for i, s in enumerate(seasons):
...     print(f"Season {i + 1}: {s}")
Season 1: Spring
Season 2: Summer
Season 3: Autumn
Season 4: Winter
```

Index 0 is the first element. The length of `positions` is 3, `3 // 2` is 1, and seat 1 is the middle. With six elements, `6 // 2` is 3: the fourth seat, the first of the second half.

`join` is the inverse of `split`: it glues a list back into one string, with your separator between the pieces. It is a method of the separator, the string before the dot, and what it returns is a string too. Leave the separator empty and the pieces touch:

```
>>> print("".join(["tic", "tac", "toe"]))
tictactoe
>>> print(" ".join(["tic", "tac", "toe"]))
tic tac toe
```

And `enumerate` is one you will use all the time. Without it, a loop that needs the position keeps its own counter, and you have to get every part of it right:

```
>>> count = 0
>>> for element in positions:
...     print(count, element)
...     count += 1
0 first
1 middle
2 last
```

`enumerate` keeps the count for you and hands you each index and its value together, so you need neither the counter nor `range(len(...))`. Give the loop one name instead of two and you see what it really hands out: a pair, which section 5 calls a tuple.

```
>>> for pair in enumerate(positions):
...     print(pair)
(0, 'first')
(1, 'middle')
(2, 'last')
```

Common mistakes. `append` changes the list and returns nothing, so assigning its result replaces your list with `None`, and the next `append` fails:

```
>>> order = []
>>> order = order.append("latte")
>>> order.append("tea")
AttributeError: 'NoneType' object has no attribute 'append'
```

Write `order.append("latte")` on a line of its own. The last index of a list of five is 4, not 5: `bestsellers[5]` on five elements raises `IndexError: list index out of range`. And a list named `list` hides Python's own `list` for the rest of the file: after `list = [1, 2]`, the call `list("espresso")` stops with `TypeError: 'list' object is not callable`. The same goes for `dict`, `set` and `str`: name a list after what it holds, `prices` or `orders`, never after its type.

Check yourself.

- **1.1** With `tables = ["t1", "t2", "t3", "t4", "t5"]`, what are `tables[-2]` and `tables[::2]`?
- **1.2** After these three lines, what are `size` and `cups`?

  ```
  cups = ["s", "m"]
  cups.append("l")
  size = cups.pop()
  ```

- **1.3** Rewrite the order loop above with `while True:` and a `break` in place of the `still_ordering` flag. Which lines go, and where does the `break` sit?

See also. Sheet 4, exercise 1. The Function reference, Lists, and Turning things into lists and back. py4e chapter 8, or Automate the Boring Stuff chapter 6 (its part on references is next week's story: read it as a preview).

2 · Lists II

Slides, pages 17 to 19.

The in-place mutation lesson

This is one of the most important pages of the week, a mistake I see often, and an exam regular. The task is to double every element of a list. The obvious loop does nothing:

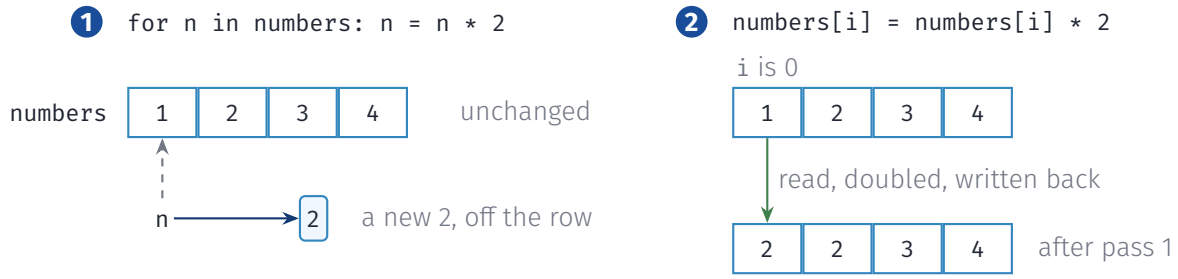
```
>>> numbers = [1, 2, 3, 4]
>>> for n in numbers:
...     n = n * 2
...
>>> print(numbers)
[1, 2, 3, 4]                  # unchanged!
```

It fails in silence, the worst kind of failure: nothing crashed and nothing warned. `n` is a name pointing at one value of the list. `n = n * 2` builds a new number and points the name `n` at it; the list is never touched. It is what week 1 showed with `x = int(x)`, and the lesson of `daily_revenue[3] * 2` in section 1 again: computing a new value does not put it in the list. Only an assignment to a seat of the list does. So ask the seat for its number, double it, and give the seat the new number:

```
>>> for i in range(len(numbers)):
...     numbers[i] = numbers[i] * 2
...
>>> print(numbers)
[2, 4, 6, 8]
```

The sentence to keep: **looping over values reads them; looping over indexes lets you change them.**

❶ In the value loop, `n` is a name on the value in seat 0. `n = n * 2` builds a new 2 off the row and moves the name onto it, and seat 0 still holds 1. ❷ In the index loop, `i` is a seat number: `numbers[i] = numbers[i] * 2` reads seat 0, doubles the 1 and writes the 2 back into the seat, so the row itself changes. Four passes later it holds `[2, 4, 6, 8]`.



The dot product is an order total

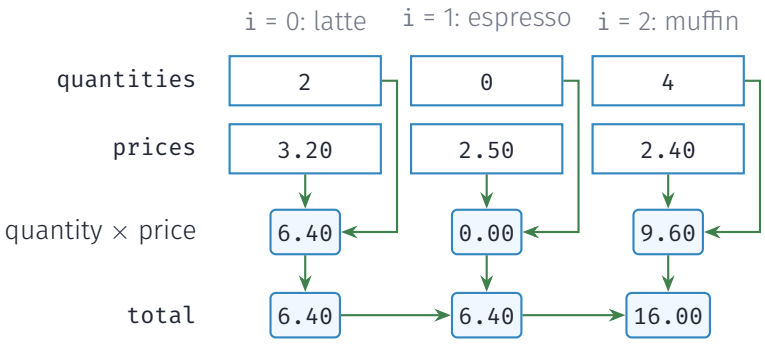
Slides, page 18. Two lists lined up by position, the quantities ordered and the prices of latte, espresso and muffin. Walk both together, multiply position by position, and add as you go:

```
>>> quantities = [2, 0, 4]          # latte, espresso, muffin
>>> prices     = [3.20, 2.50, 2.40]
>>> total = 0.0
>>> for i in range(len(quantities)):
...     total += quantities[i] * prices[i]
...
>>> print(total)
16.0
```

Read `range(len(quantities))` from the inside out, the way Python works it out: `len(quantities)` is 3, `range(3)` gives 0, 1 and 2, and the loop walks those three positions in both lists at once. The trace, one row per pass:

`i`	`quantities[i] * prices[i]`	`total` after
0	`2 * 3.20` = 6.40	6.40
1	`0 * 2.50` = 0.00	6.40
2	`4 * 2.40` = 9.60	16.00

The two rows line up seat by seat, so one index reads a quantity and its price together. Each column multiplies, and the total grows as `i` walks the three seats: 6.40, then 6.40 again, then 16.00.



Mathematicians call this a dot product; a café calls it the bill. Week 6 does the whole loop in one line with numpy, so come back to this page when it does.

When the two lists disagree

Parallel lists work only while they stay the same length, and nothing keeps them that way but you. Say a bug, or an AI assistant you trusted too fast, pops the muffin's price:

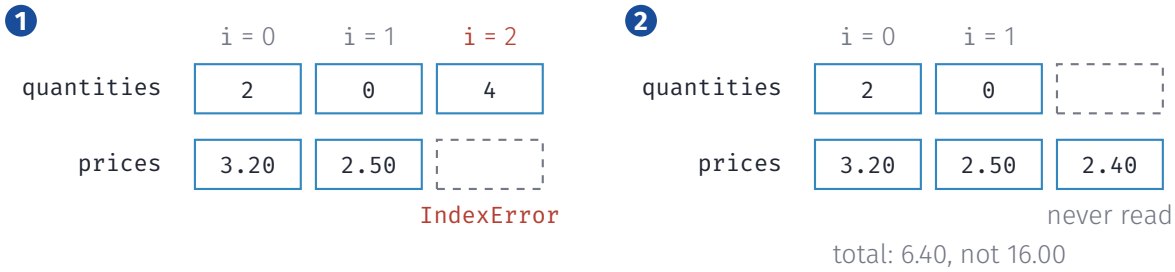
```
>>> prices = [3.20, 2.50]              # the muffin's price is gone
>>> total = 0.0
>>> for i in range(len(quantities)):
...     total += quantities[i] * prices[i]
IndexError: list index out of range
```

The loop counts three positions from `quantities`, and the third price is not there. Lose the last quantity instead and nothing crashes, which is worse:

```
>>> quantities = [2, 0]                  # the four muffins are gone
>>> prices = [3.20, 2.50, 2.40]
>>> total = 0.0
>>> for i in range(len(quantities)):
...     total += quantities[i] * prices[i]
...
>>> print(total)
6.4
```

The loop runs twice, and the muffins drop off the bill without a word: 6.40 instead of 16.00. When you walk two lists with the length of one, make sure they really are the same length.

1 With a price missing, `i` still walks 0, 1 and 2, the three seats of `quantities`, and at 2 it asks `prices` for a seat it does not have. 2 With a quantity missing, `i` walks only 0 and 1, and the muffin's price is never read: the bill comes out 6.40.



Good practice. Two lists walked in step want `zip`, which week 8 teaches: it pairs them for you, so the loop header becomes `for quantity, price in zip(quantities, prices):` with no index at all. But `zip` stops silently at the shorter list, the very failure above; since Python 3.10, `strict=True`, which no slide uses, makes it raise instead:

```
>>> list(zip([2, 0, 4], [3.20, 2.50], strict=True))
ValueError: zip() argument 2 is shorter than argument 1
```

Better still, keep each pair together from the start, as the café's order lines do.

The list toolbox

Page 19 is a reference wall of list methods, for looking up rather than learning by heart: `reverse`, `sort`, `insert`, `remove`, `count`, `+` to join two lists, `*` to repeat one, `sum` to add numbers up. Try each line once yourself; that is how they stick. Most of them change the list in place and return nothing. Two of its lines fail on purpose, and both are exam material. A list that mixes a string and a number cannot be sorted, because Python cannot decide whether `"latte"` is less than `3.20`:

```
>>> ["latte", 3.20].sort()
TypeError: '<' not supported between instances of 'float' and 'str'
```

And `remove` takes out only the **first** match, failing when there is none left:

```
>>> queue = ["bob", "ada", "ada"]
>>> queue.remove("ada")
>>> print(queue)
['bob', 'ada']
>>> while "ada" in queue:
...     queue.remove("ada")
...
>>> print(queue)
['bob']
>>> queue.remove("ada")
ValueError: list.remove(x): x not in list
```

The `while ... in` loop is the safe way to remove every match.

Good practice. Never remove from a list while a `for` walks it: the loop skips elements. Adding is no better: append inside the loop and the loop walks into what you added, and may never stop.

```
>>> queue = ["ada", "ada", "bob"]
>>> for name in queue:
...     if name == "ada":
...         queue.remove(name)
...
>>> print(queue)
['ada', 'bob']
>>> queue = ["ada", "ada", "bob"]
>>> print([name for name in queue if name != "ada"])
['bob']
```

Build a new list instead; the comprehension in section 3 does it in one line.

Common mistakes. Because `sort` changes the list and returns nothing, `queue = queue.sort()` leaves `queue` holding `None`. And the value loop above is a mistake Python never reports: if a loop was meant to change a list and the list is unchanged, check whether it loops over values.

Check yourself.

- **2.1** After `tips = [4, 6]`, which of the lines `tips[1] + 1` and `tips[1] = tips[1] + 1` changes the list, and what does `tips` hold after each?
- **2.2** Trace the order total for `quantities = [1, 3]` and `prices = [2.00, 1.50]`.

See also. Sheet 4, exercise 1 (part 3 replaces an element in place) and exercise 5b, which tracks an index across two lists. The Function reference, Lists.

3 · Comprehensions and matrices

Slides, pages 20 to 22.

List comprehensions

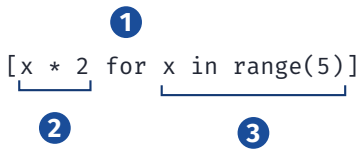
A comprehension builds a list in one line, and there are four shapes to know:

```
>>> [x for x in range(5)]                          # build
[0, 1, 2, 3, 4]
>>> [x * 2 for x in range(5)]                      # transform
[0, 2, 4, 6, 8]
>>> [x for x in range(5) if x % 2 == 0]            # FILTER (if at the END)
[0, 2, 4]
>>> ["even" if x % 2 == 0 else "odd" for x in range(5)]   # CHOOSE (if at the START)
['even', 'odd', 'even', 'odd', 'even']
```

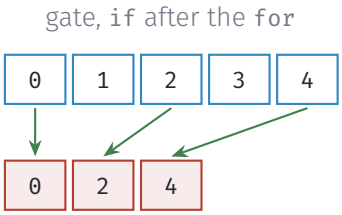
To read one, find the `for` first. Left of it is what you build: the value that goes into the new list for each element. Right of it is where the elements come from. So `[x * 2 for x in range(5)]` reads: take each `x` from `range(5)`, and put `x * 2` in the list. 0 becomes 0, 1 becomes 2, 2 becomes 4, up to 8.

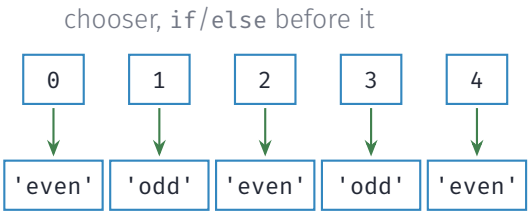
An `if` on the right of the `for` is a gate: `0 % 2` is 0, so 0 goes in; `1 % 2` is 1, so 1 stays out. An `if`/`else` on the left decides what each element becomes: every element goes in, as `"even"` or as `"odd"`. The rule that separates the two: **an `if` after the `for` filters which elements get in; an `if`/`else` before the `for` chooses what each element becomes.** That `if`/`else` on the left, `"even" if x % 2 == 0 else "odd"`, is a **ternary**: an if/else that is a value on its own. You can mix both shapes in one comprehension.

❶ Find the `for` first. ❷ Left of it is what you build, `x * 2`. ❸ Right of it is where the elements come from, `x in range(5)`. The loop that builds the same list puts each part on a line of its own: the source on the `for` line, the value inside `append`. Below, the two `if` shapes on the same five elements: the gate lets 0, 2 and 4 through, and the chooser lets all five through, each as `'even'` or `'odd'`.




```
result = []
for x in range(5):          3
    result.append(x * 2)    2
```





It looks scary the first time. Python programmers love this tool, and once you have it in your fingers it gets addictive: short to write and fast to run. Week 8 builds a whole programming style on it.

> **Good practice.** Keep a comprehension to one `for`. A filter and a choice together are fine, as the slide says; past that, the loop reads better. And a comprehension builds a list, so never use one for its side effects: `[print(x) for x in range(2)]` prints, then hands back a useless `[None, None]`. A plain `for` says what you mean.

A matrix is the whole auditorium

Put the rows back, and the auditorium of rows and seats becomes a grid of rows and columns: a list of rows, a **matrix**. This one is the week's sales, a row per day and a column per product. Two numbers find any cell, the row first, then the column, as in linear algebra.

```
>>> # columns: 0=lattes, 1=espressos, 2=muffins
>>> sales = [[12,  8, 5],       # Monday
...          [15, 11, 7],       # Tuesday
...          [22, 14, 9]]       # Wednesday
>>> print(sales[1])             # a whole row -> [15, 11, 7]
[15, 11, 7]
>>> print(sales[1][2])          # row 1, column 2 -> 7
7
>>> for row in sales:
...     print(*row)             # starred print: neat rows (Week 3's trick)
12 8 5
15 11 7
22 14 9
```

`sales[1]` is a whole row, a list itself, so the second bracket picks a seat in that row. The line breaks are for us: Python reads `[[12, 8, 5], [15, 11, 7], [22, 14, 9]]` on one line just as well. And the nesting does not stop at two: a list of lists of lists needs three indexes, and machine learning calls grids with many dimensions **tensors**.

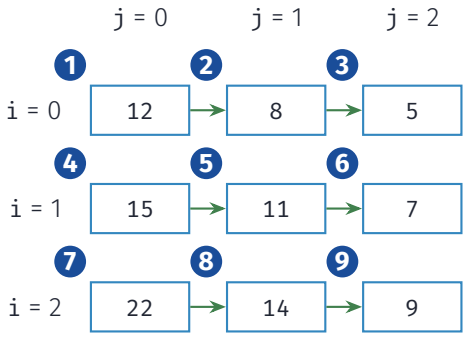
The grid as the slide draws it: a row per day, a column per product. `sales[1]` is Tuesday's whole row, and `sales[1][2]` goes on to that row's seat 2, Tuesday's muffins: the row first, then the column.



To visit every cell you need a loop at each level, an outer one over the rows and an inner one over the columns of each row. Here they print where they are:

```
>>> for i in range(len(sales)):
...     for j in range(len(sales[i])):
...         print(i, j, sales[i][j])
0 0 12
0 1 8
0 2 5
1 0 15
1 1 11
1 2 7
2 0 22
2 1 14
2 2 9
```

The numbers in the circles are the order the loops visit the cells: the inner loop, `j`, crosses a whole row before the outer loop, `i`, moves down to the next.



The inner loop walks every column of a row before the outer loop moves to the next row. To change the cells, the slide's version assigns instead of printing, `sales[i][j] += 1`, and every cell goes up by one. Week 6 hands grids like this to numpy, and pandas follows: they answer "total per product" or "the busiest day" without the nested loops. For now, the loops are yours to write and to read.

Common mistakes. In the choose shape the `else` belongs before the `for`: `[x for x in range(5) if x % 2 == 0 else "odd"]` is `SyntaxError: invalid syntax`. And a matrix takes two brackets, not one with a comma: `sales[1, 2]` raises `TypeError: list indices must be integers or slices, not tuple`.

Check yourself.

- **3.1** Turn these three lines into one comprehension:

```
line_totals = []
for i in range(len(quantities)):
    line_totals.append(quantities[i] * prices[i])
```

  Then write `[name.lower() for name in names if name != ""]` back as a loop that builds the same list with `append`.
- **3.2** In the `sales` grid above, write the index expressions for Tuesday's espressos and for Wednesday's whole row.
- **3.3** In the nested loop above, put `if j == 2:` in front of the `print`. What prints, and what do those numbers mean?

See also. Sheet 4, exercise 5. The Function reference, Lists: the comprehension and `LIST[ROW][COL]` rows.

4 · Sets

Slides, pages 24 to 27.

A set is who is in the room

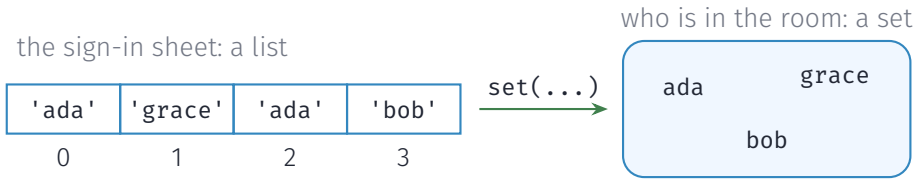
Poll. The order lives in a list. The regulars are different: no order, and nobody counts twice. What should the structure that stores them care about? Membership, whether a person is here, and uniqueness, each person once; the order they came in does not matter. That structure is the set, where this section goes.

How does a shop know you are a regular? Imagine the register appends every customer's name to a list. You come for the first time: welcome, here is 10% off your first visit. You come back the next day: welcome, here is 10% off your first visit. And again the day after. The list remembers every visit

and recognizes nobody. In real life a shop asks for something that is yours alone: a membership card, a student ID, a name.

That is the job of a set. Forget the seat numbers. At the door a sign-in sheet, a list, has Ada on it twice. A set asks another question: who is in the room? Everyone counts once and nobody has a number. So you can ask "is Ada here?", and you cannot ask who sits in seat zero.

The sign-in sheet is a list, `["ada", "grace", "ada", "bob"]`, with seat numbers and Ada twice. `set(...)` turns it into the room: everyone once, and no numbers at all.



Unique by construction

You write a set with curly braces, or call `set` on any iterable. It keeps no duplicates, keeps no order and has no index, and it is built for one question, membership:

```
>>> regulars = {"ada", "grace", "linus"}       # curly braces
>>> vowels = set("aeiou")                      # or set(any iterable)
>>> regulars.add("ada")                        # already there -> nothing happens
>>> regulars.remove("linus")
>>> print("ada" in regulars)                   # membership is what a set is for
True
```

Adding a name that is already there does nothing, which is the whole point. Removing a name that is not there crashes:

```
>>> regulars.remove("linus")       # linus has already left
KeyError: 'linus'
```

You could get uniqueness from a list: before every `append`, ask `in`, or `count` the name. That is a few lines to remember every single time. The set does it for you, by construction, and quickly.

Careful with capitals, though. `"Ada"` and `"ada"` are two different strings, so a set keeps both:

```
>>> visitors = set()
>>> visitors.add("ada")
>>> visitors.add("Ada")               # capital A: another string, another element
>>> print(len(visitors))
2
>>> visitors.remove("Ada")
>>> visitors.add("Ada".lower())       # lower() first: the same element as "ada"
>>> print(len(visitors))
1
```

That is why the café cleans every name with `.strip().lower()` before it asks the set anything.

And a set keeps no order. Run the same program three times and the same set can print in three different orders; within one run, printing a set that has not changed gives the same order every time. Build on neither: a set promises membership, never order.

The gotcha: braces around one string do not explode it. `{"aeiou"}` is a set of **one** element, the whole word; `set("aeiou")` is the five vowels. And the elements must be immutable, so strings, numbers and tuples may go in a set, and a list may not. The set itself is mutable, though: `add` and `remove` change it in place. Here is the whole ledger, an exam regular: lists, dictionaries and sets are mutable; int, float, str, bool and tuple are immutable.

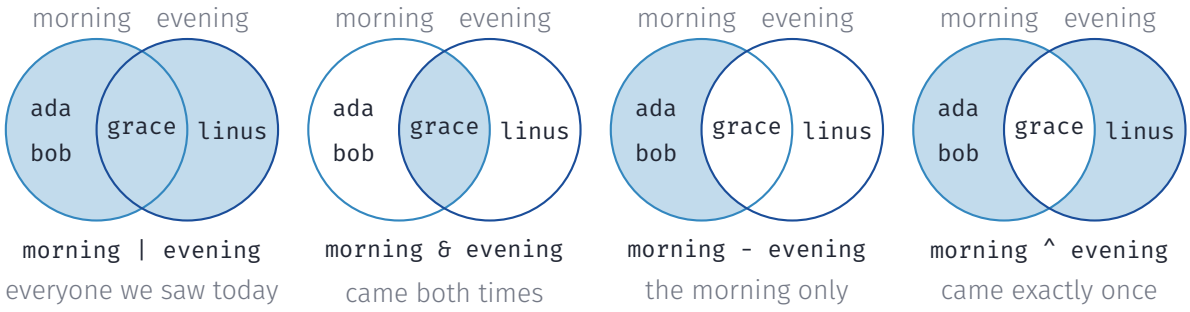
> **Good practice.** Membership in a set costs about the same however big the set grows; in a list, `in` checks the elements one by one. So when a program asks "is it in there?" again and again over a large collection, build a set once and ask the set. Week 9 measures what a dictionary lookup saves over a scan.

Set algebra

Picture two circles that overlap, the morning crowd and the evening crowd, with Grace in the overlap. Four operators answer four business questions, without a single loop:

```
>>> morning = {"ada", "grace", "bob"}
>>> evening = {"grace", "linus"}
>>> morning | evening        # union:        everyone we saw today
{'ada', 'grace', 'bob', 'linus'}
>>> morning & evening        # intersection: came BOTH times
{'grace'}
>>> morning - evening        # difference:   morning-only crowd
{'ada', 'bob'}
>>> morning ^ evening        # exclusive-or: came exactly once
{'ada', 'bob', 'linus'}
```

Each operator shades its own part of the two circles: `|` all of both, `&` the overlap, `-` the morning circle outside the overlap, and `^` both circles except the overlap.



Run these yourself and the elements may print in another order: check the contents rather than the arrangement. These operators are rare in beginners' code, and I would be happy to see them in your project, wherever they fit. Comprehensions build sets too, with braces instead of brackets: section 7's sentiment code could collect the words of a review its lexicon knows, each once, with `{w for w in review.split() if w in lexicon}`.

The regulars

Slides, page 27. Into the register it goes. When an order starts we ask for the customer's name, and if the set already has it, we welcome them back; otherwise we add it:

```
    name = input("  Customer name (ENTER to skip): ").strip().lower()
    if name != "":
        if name in regulars:
            print(f"    Welcome back, {name}!")
        else:
            regulars.add(name)
```

Order twice under the name `ada` and the second order prints `Welcome back, ada!`. The set makes duplicates impossible, so we never check whether a name is stored twice. What a set cannot tell you is how often someone came: a dictionary of visit counts could. That is the trade you make

when you choose the structure, and this week's two runnable files, `w04_regulars_set.py` and `w04_regulars_dict.py`, show both roads: the set welcomes a returning customer back, and the dict also tells them which visit it is.

Why does the café use names and not IDs? Because the café is kept simple: we assume no two customers share a name. A real shop would use a card number or a student ID, whatever makes one person unique. Improve it if you like; the café is yours too.

Common mistakes. `{}` is an empty dictionary, not an empty set: `type({})` is `<class 'dict'>`, and an empty set is `set()`. A set has no positions, so `regulars[0]` raises `TypeError: 'set' object is not subscriptable`. And a list cannot go in a set: `{[1, 2]}` raises `TypeError: cannot use 'list' as a set element (unhashable type: 'list')`; before Python 3.14 the message is just `TypeError: unhashable type: 'list'`.

Check yourself.

- **4.1** What do `{"latte"}` and `set("latte")` each hold, and why do they differ?
- **4.2** With `morning` and `evening` above, which expression gives the people who came in the evening only, and what is it?

See also. Sheet 4, exercise 3. The Function reference, Sets.

5 · Tuples

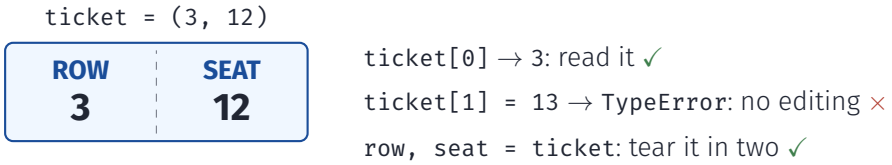
Slides, pages 28 to 30.

A tuple is a ticket

Every seat in the hall has a ticket, row 3, seat 12, printed: `ticket = (3, 12)`. You can read it, `ticket[0]` is 3. You cannot edit it. And you can tear it in two: `row, seat = ticket` gives two names in one line.

Think of a cinema. It sells row 3, seat 12 once, otherwise two people sit on top of each other, and once you hold that ticket you cannot decide you would rather sit in seat 14. A list would let you; a tuple will not. Our order lines are tickets too. The customer asked for two lattes, they get two lattes, and the receipt says two. If something later in the program could turn that 2 into a 10, a bug, a stray `pop`, a hurried fix, you would bill ten lattes for the two you served. `("latte", 2)` is a fact about the bill, and a fact should not change after you record it.

Drawn, the ticket lets you read it, refuses an edit with a `TypeError`, and lets you tear it in two, so what it says stays what it said.



Lists that promise not to change

```
>>> line = ("latte", 2)              # parentheses... optional!
>>> line2 = "muffin", 4              # the COMMA makes the tuple
>>> empty = ()
>>> single = "espresso",             # singleton: note the comma!
>>> not_a_tuple = ("espresso")       # <- this is just a STRING in parentheses
>>> line[0]                          # ordered + indexable, like a list
'latte'
>>> type(single)
<class 'tuple'>
>>> type(not_a_tuple)
<class 'str'>
```

The comma makes the tuple, not the parentheses, which is why a one-element tuple needs a trailing comma; `type` settles any doubt. A tuple is ordered and indexable like a list, and its immutability is the point: it is what lets a tuple sit in a set and, in the next section, in front of a dictionary's colon, where a list may not.

A tuple protects what is inside it, not the name: assign to the name again and Python lets you, because a name can point at a new tuple whenever you like. What a tuple refuses is a change inside:

```
>>> ticket = (3, 12)
>>> ticket[1] = 13                   # change one value on the ticket: refused
TypeError: 'tuple' object does not support item assignment
>>> ticket = (3, 14)                 # a whole new ticket: allowed
>>> print(ticket)
(3, 14)
```

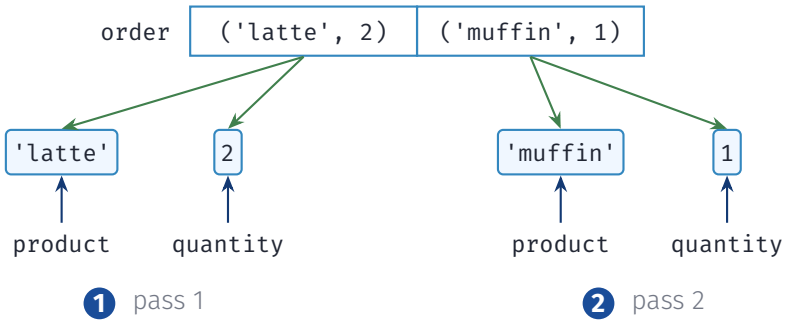
Good practice. Unpack a tuple by name, `for product, quantity in order:`, rather than reaching in by position, `line[0]` and `line[1]`: names say what each field is, positions make the reader remember. And choose a tuple over a list whenever the number and the meaning of the fields are fixed.

Quick fire

Poll. Can a tuple be added to a set, and why: yes, tuples are immutable; no, only strings and numbers; yes, anything goes in a set; or no, sets reject containers? Yes, because tuples are immutable. The rule is about change, not containers: a tuple holds several values and still goes in, `s.add((7, 8, 9))`, because it cannot change. A list can, so `s.add([7, 8, 9])` raises a `TypeError` that calls the list unhashable.

Page 30's other three predictions. `(1, 2, 3) in ((1, 2, 3), (4, 5, 6))` is `True`, because that exact tuple is one of the two elements. `b, c = a` unpacks a pair into two names, and `b, c = c, b` swaps two names in one line, with no temporary variable. And `sorted` takes any iterable, so `sorted(set("abcde"), reverse=True)` hands back the list `['e', 'd', 'c', 'b', 'a']`. The same unpacking is what the receipt loop in `cafe_w4.py` does when it writes `for product, quantity in order:`: each tuple splits into two names as it comes out.

With the order from the receipt above, `[("latte", 2), ("muffin", 1)]`, the loop hands out one ticket per pass and tears it in two as it comes out. ❶ Pass 1 puts `product` on `"latte"` and `quantity` on 2. ❷ Pass 2 puts them on `"muffin"` and 1.

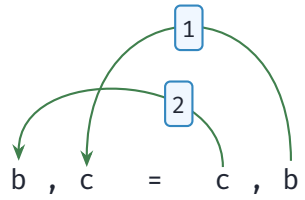


The swap deserves a second look, because it is a classic. Try it in two lines and one value is lost:

```
>>> b, c = 1, 2
>>> b = c                  # b is 2 now, and the 1 is gone
>>> c = b
>>> print(b, c)
2 2
>>> b, c = 1, 2
>>> b, c = c, b            # both old values are read before either name changes
>>> print(b, c)
2 1
```

Read the one-line swap in two steps. ❶ Python reads the whole right-hand side first: `c` is 2 and `b` is 1, and both values are in hand before any name changes. ❷ Then it assigns, left to right: `b` takes the 2 and `c` takes the 1. The two lines go wrong for the opposite reason: `b = c` moves `b` onto the 2 at once, the 1 is left with no name, and `c = b` finds only the 2.

one line: both values kept

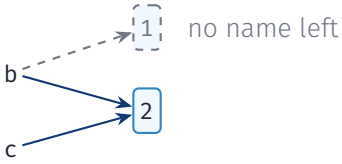


❶ read the right side first: `c` is 2, `b` is 1

❷ then assign: `b` takes the 2, `c` the 1

two lines: one value lost

```
b = c
c = b
```



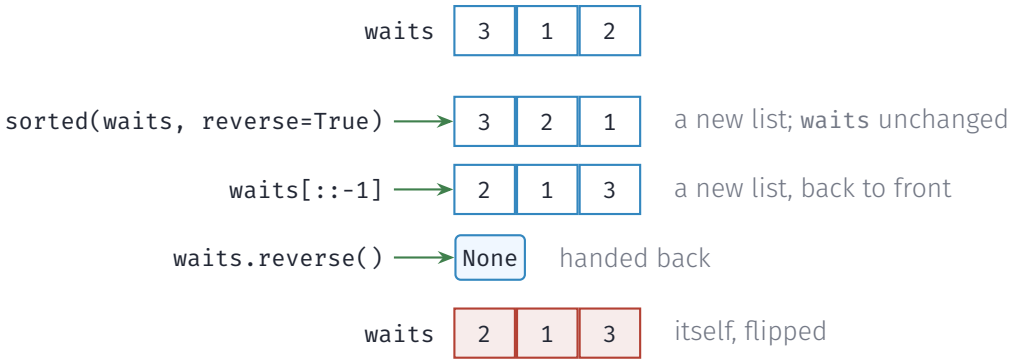
`print(b, c)` shows `2 2`

What tuples have to do with it is the question of sheet 4, exercise 2.

And careful with `reverse=True`: it does not flip the list, it sorts it in descending order. To flip the order a list has now, `[::-1]` hands you a new list back to front, and `.reverse()` flips the list itself:

```
>>> waits = [3, 1, 2]
>>> print(sorted(waits, reverse=True))    # sorted, largest first
[3, 2, 1]
>>> print(waits[::-1])                    # a new list, back to front
[2, 1, 3]
>>> waits.reverse()                       # flips waits itself, and hands back None
>>> print(waits)
[2, 1, 3]
```

Three roads from one list. `sorted(waits, reverse=True)` builds a new, sorted list and leaves `waits` alone. `waits[::-1]` builds a new list back to front. `waits.reverse()` changes `waits` itself and hands back `None`.



Common mistakes. `("espresso")` is a string in parentheses, not a tuple. And a swap with no spare name needs one line: in two, the first assignment overwrites a value the second still needs.

Check yourself.

- **5.1** Which of `a = (4)`, `b = (4,)`, `c = ()` and `d = 4,` leave a tuple in their name?
- **5.2** Why can a tuple be a dictionary key when a list cannot?
- **5.3** The order loop appends `(product, quantity)` as one element. What would go wrong if it appended the product and the quantity as two separate elements?

See also. Sheet 4, exercise 2. The Function reference, Tuples. py4e chapter 10.

6 · Dictionaries

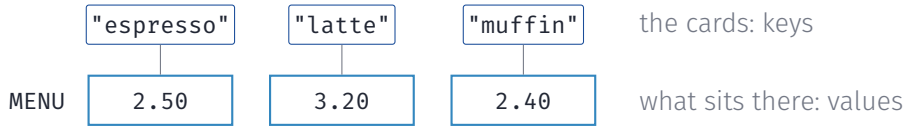
Slides, pages 31 to 35.

A dict is a row of reserved seats

Look at `bestsellers` as we first made it. Why is latte in seat 0 and tea in seat 4? No reason: that is the order they came to mind, and now you have to remember it every time you ask. A list's indexes are handed out by Python, 0, 1, 2, and they carry no meaning. A dictionary lets you name the seats yourself. Picture a committee on a row of chairs: nobody says "chair 0", they say the president, the vice-president, the treasurer.

So put a name card on every seat. Page 31 draws three seats, espresso, latte and muffin, holding 2.50, 3.20 and 2.40. As a list, that row is `MENU`, and you ask for seat two: `MENU[2]` gives 2.40. With the cards on, you ask for the seat marked muffin instead, `MENU["muffin"]`, and you get the same 2.40. That is a **dictionary**: the key is an index with a name, the card, and what sits in the seat is the value. One card per seat, so keys are unique, and a card never changes. This room's seating chart is a dictionary keyed by tickets: `{(3, 12): "ada"}`.

Page 31's row, drawn: each seat carries a card, `"espresso"`, `"latte"` or `"muffin"`, and you ask for a seat by its card. The cards are the keys, and what sits in each seat is its value.

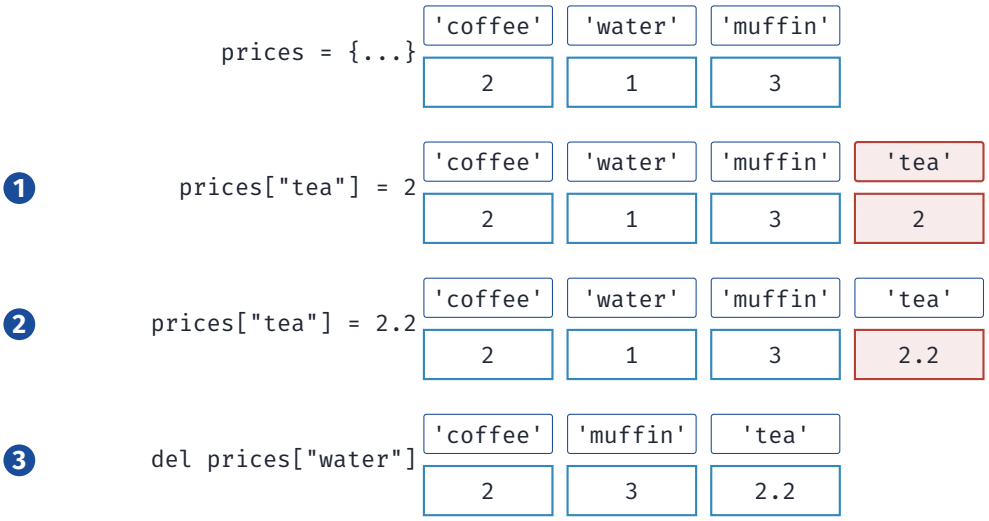


Lists with meaningful indexes

A list answers "what is at position 3?". A dictionary answers "what is the price of a latte?".

```
>>> prices = {"coffee": 2, "water": 1, "muffin": 3}     # key: value pairs
>>> print(prices["muffin"])                             # look up BY KEY
3
>>> prices["tea"] = 2                                   # add: assign to a new key
>>> prices["tea"] = 2.2                                 # replace: assign to an old key
>>> del prices["water"]                                 # remove an entry
>>> print("coffee" in prices)                           # membership tests KEYS
True
```

The same dictionary after each line. (1) A new key adds a card at the end. (2) An old key keeps its place and takes the new value. (3) `del` takes the card out with its seat.



Keys must be immutable and unique: strings, numbers, tuples. Values can be anything, and that cuts both ways. Watch `"tea"` in the block: it starts as 2 and becomes 2.2, and nothing would stop it from becoming text next. A dictionary is mutable like a list, and just as permissive, so keep track of what your values hold.

Comprehensions work here too, with the same reading rule. Find the `for`: left of it, the key, a colon and the value; right of it, where the keys come from. Here every product starts a fresh tally at zero:

```
>>> {product: 0 for product in ["latte", "tea", "muffin"]}
{'latte': 0, 'tea': 0, 'muffin': 0}
```

Asking a dict about itself

```
>>> prices = {"coffee": 2, "water": 1, "muffin": 3}
>>> print("Items:", *prices.keys())
Items: coffee water muffin
>>> print("Total:", sum(prices.values()))
Total: 6
```

`keys()` hands you the keys, `values()` the values, and `sum` of the values totals them, a stock-take or a revenue total in one expression. Looping over a dictionary loops over its keys, and you look each value up inside the loop.

Look at the order of that first output: coffee, water, muffin, exactly the order the keys were typed. That is a promise, not a coincidence: **a dictionary keeps its keys in the order they were first added.** And the half people get wrong: replacing the value of a key that is already there does not move the key to the end. It stays where it was.

The menu becomes data

Page 34. This is the second debt, and the moment the week builds toward. Last week, pricing was `price_of`, an if/elif chain of fifteen lines, two more for every new product:

```
def price_of(product):
    if product == "espresso":
        return 2.50
    elif product == "latte":
        return 3.20
    ...
```

This week the menu is two lines of data:

```
MENU = {"espresso": 2.50, "latte": 3.20, "cappuccino": 3.00,
        "croissant": 2.80, "muffin": 2.40, "tea": 2.20}
```

`MENU[product]` replaces the whole function, and `product not in MENU` replaces the old 0.0 signal for a product we do not sell. Stock becomes a second dictionary the same way. Open `cafe_w3.py` beside `cafe_w4.py` and you see the chain and the two lines together; the register uses them in two places, `elif product not in MENU:` and `MENU[product] * quantity`. Pricing a product used to take two lines of code; now it takes one line of data. Code is what you test and debug; data is what you type.

And data can change while the program runs, which the chain never could. You start selling paninis: one assignment to `MENU` and one to `stock`, because the register looks a product up in both; skip the second and the first panini order stops it with `KeyError: 'panini'`. The muffins sell out at noon: one `del`. A new price: assign to the old key.

```
>>> MENU["panini"] = 4.50          # a new product, while the register runs
>>> del MENU["muffin"]             # sold out for today
>>> print("muffin" in MENU)
False
```

From then on, `product not in MENU` turns the muffin order away, and nothing else in the code changes.

Good practice. Choose between `MENU[product]` and `MENU.get(product)` on purpose. Square brackets fail loudly on a missing key, which is right for a price: a product without one is a bug you want to hear about. `.get` returns `None`, or the default you give it, which is right when absence is normal. And walk the pairs directly with `.items()`, which the slides do not cover and the library policy allows:

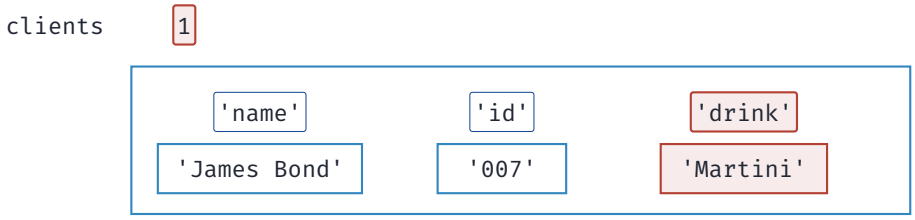
```
>>> specials = {"mocha": 3.60, "chai": 3.10}
>>> for product, price in specials.items():
...     print(f"{product:<12}{price:>6.2f}")
mocha         3.60
chai          3.10
```

Nested dicts

A dictionary's values can be dictionaries, and you reach inside by chaining the keys, the outer one first. Page 35's `clients` holds one dictionary per client, `clients = {1: {"name": "James Bond", "id": "1"}}`, so `clients[1]["name"]` is `James Bond`; `clients[1]["id"] = "007"` replaces a field and `clients[1]["drink"] = "Martini"` adds one, so `print(clients[1])` shows `{'name': 'James Bond', 'id': '007', 'drink': 'Martini'}`. It is the matrix idea again, with names instead of numbers: `clients[1]["drink"]` reads "client 1, field drink", the way `sales[1][2]` reads "row 1, column 2". And asking for a key that is not there adds a word to your error vocabulary, the dictionary's version of `IndexError`:

```
>>> clients = {1: {"name": "James Bond", "id": "1"}}
>>> print(clients[0])
KeyError: 0
```

Drawn after the two changes, the outer dictionary has one card, `1`, and its seat holds a whole dictionary of its own. `clients[1]` goes through the outer card to that inner dictionary, and `["drink"]` goes on to the inner card and its `'Martini'`. There is no card 0, so `clients[0]` stops at the first step.



`clients[1]["drink"]`: the outer card 1, then the inner card `'drink'`

`clients[0]`: no card 0, so `KeyError: 0`

Common mistakes. A missing key stops the program: with `prices` above, `prices["cake"]` raises `KeyError: 'cake'`. Ask first with `"cake" in prices`, or use `prices.get("cake", 0)`, which returns 0 when the key is missing. And a key cannot appear twice: `{"latte": 3.20, "latte": 3.50}` keeps only the last value, `{'latte': 3.5}`.

Check yourself.

- **6.1** What does this print?

```
sizes = {"small": 1.0, "large": 1.5}
sizes["large"] = 1.75
sizes["medium"] = 1.25
print(sizes)
```

- **6.2** With `prices = {"coffee": 2, "muffin": 3}`, what does `prices[0]` raise, and why is it not an `IndexError`?
- **6.3** After this line, what do `"small" in sizes` and `1.0 in sizes` give, and why?

```
sizes = {"small": 1.0, "large": 1.5}
```

See also. Sheet 4, exercise 4. The Function reference, Dictionaries. py4e chapter 9, or Automate the Boring Stuff chapter 7.

7 · Case study: what do our customers think?

Slides, pages 36 to 40.

The ask and the plan

The owner's question, in plain English: we have a file of customer reviews; which products do people love, and which are letting us down? That is sentiment analysis, pulling a feeling out of free text. The professional version uses machine learning; the useful version needs only this week's tools. Decompose before writing anything:

1. Load a lexicon, words with weights, from a file into a dictionary.
2. For each review, split the text into words, look each word up, and add the weights.
3. Average per product and report.

Then draw it as an activity diagram and write the pseudo-code before you build: the week-3 method, and also how you explain a plan to your group.

> **In the project.** This skill comes back with the project's news: each market comes with a file of daily headlines, and the dataset guide describes them and the project's word list. Loading the files belongs to the project's data piece in the brief, and what reaches your agent, and how, is the agent contract's to say: read it before you score anything. Trading on the scores is optional, and the competition rewards it. You build the scorer yourself, with only what the library policy lists.

What follows is the starting point, the most basic version there is, and you can make it much smarter.

Step 1: the lexicon file into a dict

The lexicon is a plain text file, one word and its weight per line: `love,0.9`, `loved,0.9`, down to `terrible,-0.9`. Count the pieces you already own in `load_lexicon`: an empty dict, week 3's file loop, this week's `split`, and `float`, because everything read from a file arrives as a string.

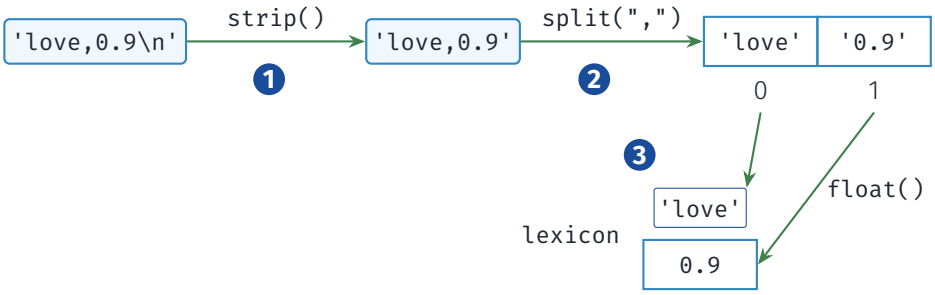
```
DATA_DIR = "data/"

def load_lexicon():
    """Read the sentiment lexicon (word,weight per line) into a dict."""
    lexicon = {}
    for line in open(DATA_DIR + "lexicon.txt"):
        parts = line.strip().split(",")
        lexicon[parts[0]] = float(parts[1])
    return lexicon
```

The two files, `lexicon.txt` and `reviews.txt`, go in a `data/` folder beside the program, because that is where `DATA_DIR` points. Make it a habit: code in one place, data in another.

Walk the first line through the loop. `line` is `'love,0.9\n'`, the text and its newline. Python works out everything right of the `=` first. `strip` cleans both ends of the line: the spaces, the tabs, and the newline every line read from a file ends with. `split(",")` cuts what is left, `'love,0.9'`, at the comma, giving `['love', '0.9']`. Seat 0 becomes the key and seat 1, turned into a number, the value: `lexicon["love"]` is now 0.9. The loop does the same for every line of the file.

❶ `strip` takes the newline off. ❷ `split(",")` cuts at the comma, and the two pieces land in seats 0 and 1. ❸ Seat 0 becomes the key, and seat 1, through `float`, the value.



Why weights rather than good or bad: "terrible" should hurt more than "dry", and a thumbs up or down cannot say so. The number carries the strength.

> **Good practice.** Keep data out of the code. The lexicon lives in a file, so a new word is a one-line edit with no code change, and the menu could live in one too. Keep the folder in one constant, `DATA_DIR`, and build every path from it, so moving the data changes one line. And clean text once, where it enters (`strip`, `lower`), so the rest of the program compares like with like.

> **In the project.** The project's nine training files go in a `data/training/` folder of their own; the Project code and data folder shows the whole tree.

Step 2: score every review

Inside `review_scores`, each review line goes through three moves:

```
        product, text = line.split(",", 1)
        score = 0.0
        for word in text.lower().split():
            if word in lexicon:
                score = score + lexicon[word]
        if product in totals:
            totals[product] = totals[product] + score
            counts[product] = counts[product] + 1
        else:
            totals[product] = score
            counts[product] = 1
```

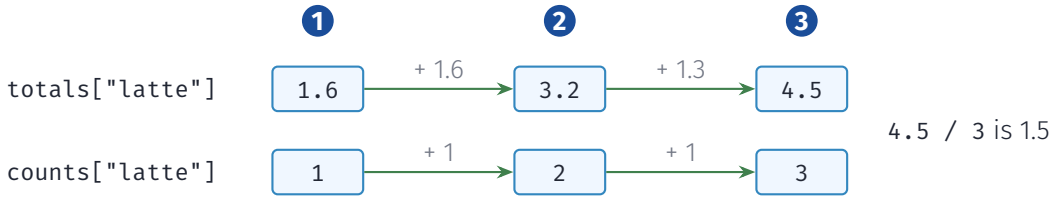
First, `split(",", 1)` splits only once: its second argument is how many cuts it may make. The product name comes off the front and the review keeps its own commas. Second, the score adds the weight of every word the lexicon knows, after `lower`, because the lexicon only knows lowercase words; unknown words add nothing, so the lexicon acts as a filter. Third, the counting pattern, which you will use for the rest of the course: if we have seen this product, add to its total and its count; otherwise start both. That is how you accumulate grouped totals without knowing the groups in advance. A second pass divides each total by its count.

Walk the first latte review through it by hand: "the latte was excellent and the milk was perfect".

```
word        in the lexicon?   score after
the         no                0.0
latte       no                0.0
was         no                0.0
excellent   yes, 0.8          0.8
and         no                0.8
the         no                0.8
milk        no                0.8
was         no                0.8
perfect     yes, 0.8          1.6
```

Latte has three reviews. This one scores 1.6; "love this latte best in town" scores 0.9 + 0.7 = 1.6; "excellent latte friendly staff" scores 0.8 + 0.5 = 1.3. The average is 4.5 / 3 = 1.5, the latte's number in the verdict below.

Follow the latte's card in both dictionaries. ❶ Its first review starts both: `totals["latte"]` is 1.6 and `counts["latte"]` is 1. ❷ The second adds its score and one review: 3.2 and 2. ❸ The third: 4.5 and 3. The second pass divides one by the other, and 4.5 / 3 is 1.5.



The verdict

Page 39. Run `cafe_w4.py`, give any cashier name and choose 4, Reviews:

```
Average review score per product: {'latte': 1.5, 'espresso': 0.53, 'croissant': 1.2, 'muffin': -0.83, '
    ↪ cappuccino': 0.7, 'tea': -0.15}
```

The lattes are loved, the muffins are the problem, and the tea is below zero too: free text, turned into numbers a manager can act on. The code rounds each average to two decimals with `round`, which is why espresso shows 0.53 and not a long tail of digits.

Now ask whether you believe espresso's 0.53. Its three reviews:

Review	What the lexicon finds	Score
great espresso but the service was slow	great 0.6, slow -0.5	0.1
burnt espresso not great	burnt -0.7, great 0.6	-0.1
perfect espresso rich and smooth	perfect 0.8, rich 0.4, smooth 0.4	1.6

Read the second one as a person and it is a bad review. The program sees "great" and misses the "not": this bag-of-words scoring is crude. Flip the sign of a word that follows "not", and that review scores -1.3, and espresso's average falls from 0.53 to 0.13. The arithmetic was right; the reading was wrong, and you only find that by going back to the reviews. It's a prototype: it gets us somewhere, and the number is not perfect, so dig in until you understand why it behaves as it does. Flag this one for your project.

How would we know the lexicon works? Score texts whose verdict you already know and count the hits. Our reviews carry no labels, but your project's data has one built in: the next day's price.

Rapid prototyping, and a shortcut

What we just did has a name, **rapid prototyping**: a few lines of Python that test an idea before anyone spends money building it properly. You start with something rough and iterate, and iterate, until it kind of works. Then the concept is proven, and two roads open: build the full program in Python, or hand the proven idea to another language, Java or C#. Python is very good at that first part, which is one reason it is used so much.

Iterating also means less hard-coding as you go. The menu is still written into the code, so a new price means editing the program, and a typo means editing it again. Load the menu from a file at startup, the way the lexicon is loaded, and a new price is a change to a file.

And the counting pattern has a one-line form you will meet in the wild, one line per dictionary:

```
totals[product] = totals.get(product, 0) + score    # the .get() one-liner
counts[product] = counts.get(product, 0) + 1
```

`get` returns the stored value, or 0 when the product is not there yet, and nothing crashes. Swap in both lines, never the first alone: the averages divide by `counts`. I like this one a lot, and you do not have to use it: use whichever you read faster.

Common mistakes. Forget `float` and the weight stays text, so adding it fails: `0.0 + "0.9"` raises `TypeError: unsupported operand type(s) for +: 'float' and 'str'`. Forget `.lower()` and "Love" at the start of a review never matches the lexicon's "love": the word silently scores nothing. And a word with punctuation stuck to it, `great,` with its comma, is a different string from `great`, so the lexicon misses it too: the comma puzzle from section 1.

Check yourself.

- **7.1** Why does the scoring loop use `split(",", 1)` rather than `split(",")`?
- **7.2** With the lexicon `{"great": 0.5, "cold": -0.25}`, what does the review `"great coffee but cold"` score?

See also. Sheet 4, exercise 6: the morale scorer is this case study on new data. The Function reference, Dictionaries: the `.get` row.

Where the café is now

Slides, pages 41 to 44.

The menu and the stock are dictionaries, the orders are lists of tuples, so the receipt lists every line again, the regulars are a set, and the café scores its own reviews. One fragility stays on purpose, because it is next week's story. At the quantity prompt, type the word `three` instead of the digit, and the register dies:

```
ValueError: invalid literal for int() with base 10: 'three'
```

`int()` could not read the word, and nothing caught the error, so one keystroke ended the program. Press Enter by accident and you get the same error with nothing between the quotes, `''`. Picture it: you have been selling for three hours, every order and all the stock are in memory, you burn your hand on a coffee, you hit Enter at the quantity prompt, and the day is gone. Next week: errors, exceptions, and a register that refuses to die.

Read `cafe_w4.py` this week, all of it. Apart from its last two lines, the guard week 5 explains, it uses only the tools we have covered, so any other line that puzzles you is a gap worth closing now, before the weeks that build on it. The practical sheets use the same tools on a different program, which makes the café a worked example for them.

What the exam can ask from this week, as page 42 lists it: predict list operations, rewrite an index loop as a comprehension and back, trace the in-place mutation trap, tell the four structures apart and justify the choice, and walk the sentiment loop by hand on three lines of input.

The checklist on page 44 is your revision list. Anything you cannot tick, read its section here again, then do the matching exercise on sheet 4. Self-check 4 asked the kinds of questions the exam asks, and every week builds on the one before, so a gap here costs you again next week.

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the outputs in the blocks of sections 1 and 2, errors included, and the pictures beside them, and write each one exactly, brackets and quotes included; then look.
- Trace the order total for `quantities = [3, 1, 2]` and `prices = [2.50, 3.20, 2.40]` in a table like page 18's, then run it to check. Then write, from memory, the loop that fails to double a list and the one that works, and say in one sentence why the first fails.
- For each of the four structures, write one sentence on the question it answers; then compare with sections 1 and 4 to 6. Then name the structure for three needs of your own and defend each choice in a sentence.
- Walk the three muffin reviews through `review_scores` by hand, with the lexicon file open: each review's score, then `totals["muffin"]` and `counts["muffin"]` after each line, then the average. Option 4 shows -0.83.
- Write `initials(names)`, which returns each name's first letter in capitals, once with `append` and once as a comprehension; run both to check: `initials(["ada", "grace"])` gives `['A', 'G']`.
- In a few days, answer the check-yourself questions again without these notes, then take the week's self-check by hand, with Python closed.

Answers to check yourself

- **1.1** `'t4'` and `['t1', 't3', 't5']`: index -2 is the second from the end, and a step of 2 takes every other seat.
- **1.2** `size` is `'l'` and `cups` is `['s', 'm']`: `pop` removes the last element, the one `append` just added, and hands it back.
- **1.3** `while True:` replaces both `still_ordering = True` and `while still_ordering:`, and `break` replaces `still_ordering = False`, under `if product == "done":`. The rest of the loop stays as it is.
- **2.1** Only the second. The first works out 7 and keeps it nowhere, so `tips` stays `[4, 6]`; the second assigns to the seat, so `tips` becomes `[4, 7]`.
- **2.2** `total` starts at 0.0; `1 * 2.00` makes it 2.0; `3 * 1.50` adds 4.5, so 6.5.
- **3.1** What the loop appends goes left of the `for`:

```
line_totals = [quantities[i] * prices[i] for i in range(len(quantities))]
```

  Back, an `if` after the `for` becomes an `if` inside the loop:

```
cleaned = []
for name in names:
    if name != "":
        cleaned.append(name.lower())
```

- **3.2** `sales[1][1]`, which is 11, and `sales[2]`, which is `[22, 14, 9]`: the row first, then the column.
- **3.3** Three lines, `0 2 5`, `1 2 7` and `2 2 9`. `j` is the column, and column 2 is muffins: the muffins sold on Monday, Tuesday and Wednesday.
- **4.1** `{'latte'}` holds one element, the whole word: braces do not explode a string. `set("latte")` holds the letters l, a, t and e, each once, in whatever order the set prints them.
- **4.2** `evening - morning`, which is `{'linus'}`.
- **5.1** `a` holds the number 4, because parentheses alone only group. `b`, `c` and `d` hold tuples: the comma makes `(4,)` and `4,` tuples, and `()` is the empty one.
- **5.2** A key must never change, and a tuple is immutable while a list is not.
- **5.3** The order would become one flat list of products and numbers, and nothing would tie each quantity to its product any more. One tuple per line keeps the pair together, which is what the receipt loop unpacks.
- **6.1** `{'small': 1.0, 'large': 1.75, 'medium': 1.25}`: replacing a value keeps the key where it was, and a new key goes at the end.
- **6.2** `KeyError: 0`. A dictionary has no positions: the 0 is looked up as a key, and there is no key 0.
- **6.3** `True` and `False`: `in` on a dictionary looks at the keys, never at the values.
- **7.1** The review text can contain commas. Splitting once takes the product name off the front and keeps the rest of the sentence whole.
- **7.2** `0.25`: "great" adds 0.5 and "cold" adds -0.25; "coffee" and "but" are not in the lexicon and add nothing.